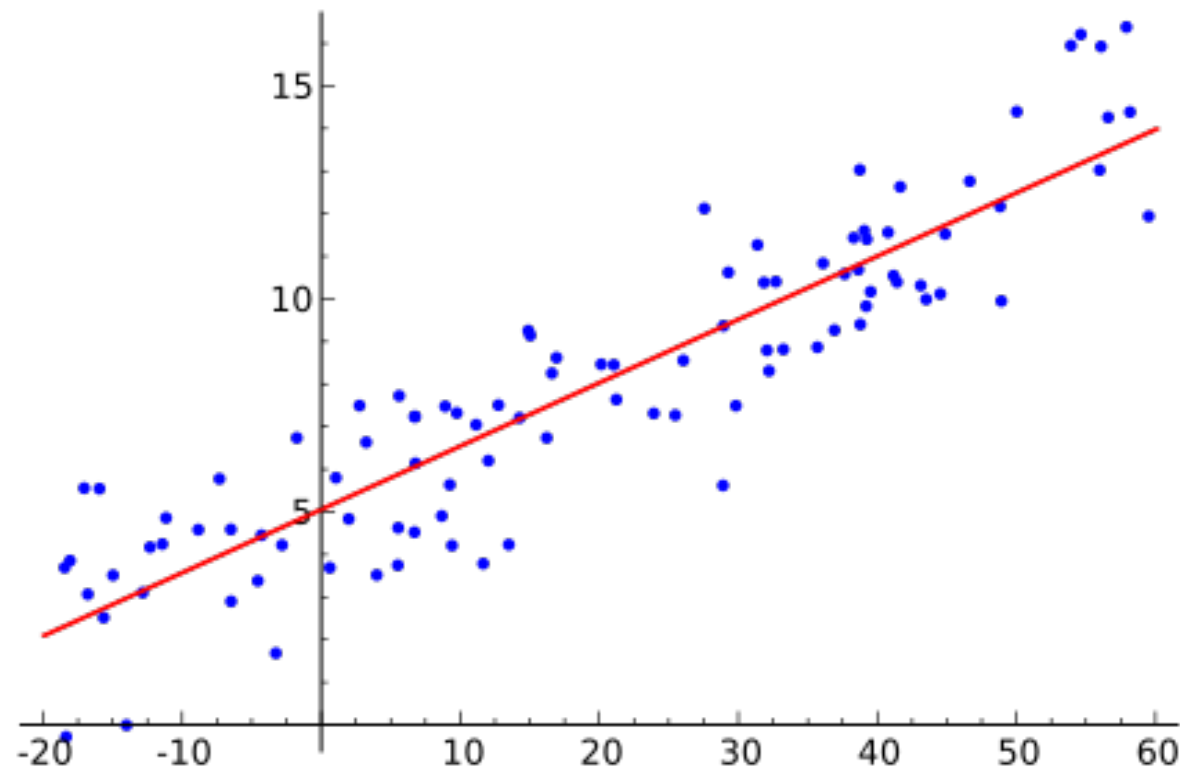


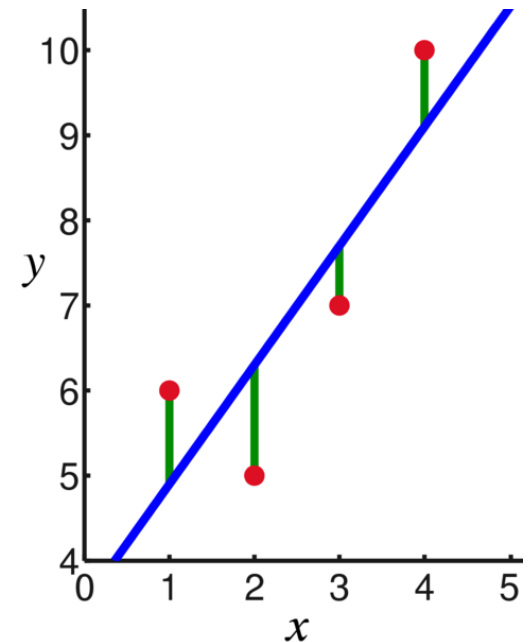
Chapter 2

Least squares problems



Linear curve fitting

- **Notation:** n objects at locations $\mathbf{x}_i \in \mathbb{R}^p$. Every object has measurement $y_i \in \mathbb{R}$.
- **Approximate** “regression targets” y as a **parametrized function** of x .
- Consider a 1-dim problem initially.
- Start with n data points (x_i, y_i) , $i = 1, \dots, n$.
- Choose d **basis functions** $g_0(x), g_1(x), \dots$.
- Fitting to a **line** uses **two** basis functions $g_0(x) = 1$ and $g_1(x) = x$. In most cases $n \gg d$.
- **Fit function = linear combination of basis functions:**
$$f(x; \mathbf{w}) = \sum_j w_j g_j(x) = w_0 + w_1 x.$$
- $f(x_i) = y_i$ exactly is (usually) **not possible**, so approximate $f(x_i) \approx y_i$
- n **residuals** are defined by $r_i = y_i - f(x_i) = y_i - (w_0 + w_1 x_i)$.

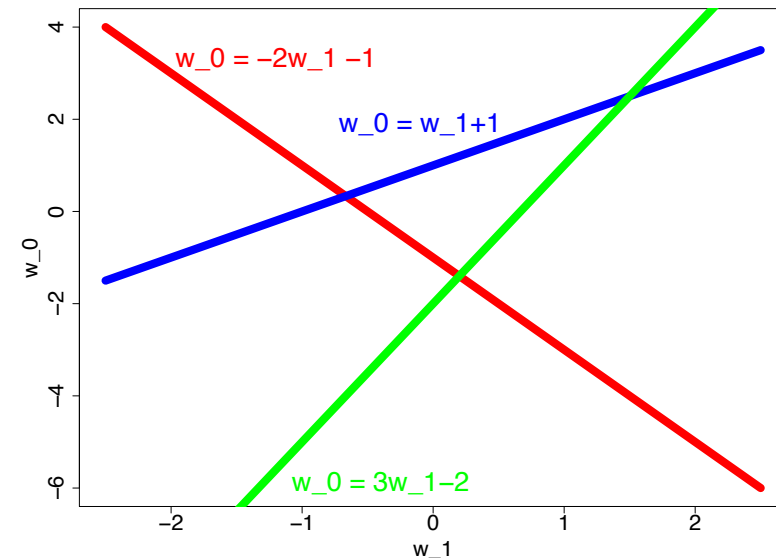
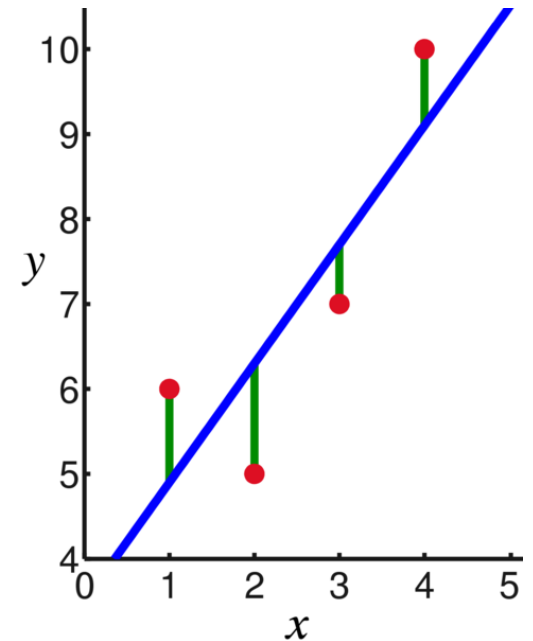


Calculus or algebra?

- **Quality of fit** can be measured by **residual sum of squares**

$$RSS = \sum_i r_i^2 = \sum_i [y_i - (w_0 + w_1 x_i)]^2.$$

- Minimizing RSS with respect to w_1 and w_0 provides the **least-squares fit**.
- To solve the **least squares problem** we can
 1. set the derivative of RSS to zero
 $\rightsquigarrow$ **calculus**, or
 2. solve an **over-determined system**
 $\rightsquigarrow$ **algebra**: $w_0 + w_1 x_i = y_i, i = 1, \dots, n$
- The results you get are...
 - **mathematically the same**, but
 - have **different numerical properties**.



Matrix-vector form

- Write $f(x) \approx y$ in matrix-vector form for n observed points as

$$\underbrace{\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}}_X \underbrace{\begin{bmatrix} w_0 \\ w_1 \end{bmatrix}}_w \approx \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}}_y$$

- We minimize the sum of squared errors, which is the squared norm of the residual vector $\mathbf{r} = \mathbf{y} - X\mathbf{w}$:

$$RSS = \sum_{i=1}^n (y_i - (X\mathbf{w})_i)^2 = \|\mathbf{y} - X\mathbf{w}\|^2 = \|\mathbf{r}\|^2 = \mathbf{r}^t \mathbf{r}.$$

- $RSS = 0$ only possible **if all the data points lie on a line.**

Basis functions

X has as many columns as there are basis functions. Examples:

- **High-dimensional linear functions**

$\mathbf{x} \in \mathbb{R}^p$, $g_0(\mathbf{x}) = 1$ and $g_1(\mathbf{x}) = x_1, g_2(\mathbf{x}) = x_2, \dots, g_p(\mathbf{x}) = x_p$.

矩阵 X 的第 i 行, 就是把第 i 个数据点 $\mathbf{x}_i$ 放入基函数 (转置后) 得到的行向量

$\mathbf{X}_i = \mathbf{g}^t(\mathbf{x}_i) = (1, \mathbf{x}_i^t)$, (i -th row of X)

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^t \mathbf{g} = w_0 + w_1 x_1 + \dots + w_p x_p.$$

- **Document analysis:** Assume a fixed collection of words:

$\mathbf{x}$ = text document

$$g_0(\mathbf{x}) = 1$$

$$g_i(\mathbf{x}) = \#(\text{occurrences of } i\text{-th word in document})$$

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^t \mathbf{g} = w_0 + \sum_{i \in \text{words}} w_i g_i(\mathbf{x}).$$

Solution by Calculus

$$\begin{aligned}RSS &= \mathbf{r}^t \mathbf{r} = (\mathbf{y} - X\mathbf{w})^t (\mathbf{y} - X\mathbf{w}) \\&= \mathbf{y}^t \mathbf{y} - \mathbf{y}^t X\mathbf{w} - \mathbf{w}^t X^t \mathbf{y} + \mathbf{w}^t X^t X \mathbf{w} \\&= \mathbf{y}^t \mathbf{y} - 2\mathbf{y}^t X\mathbf{w} + \mathbf{w}^t X^t X \mathbf{w}.\end{aligned}$$

Minimization: set the gradient (vector of partial derivatives) to zero:

$$\nabla_{\mathbf{w}} RSS = \frac{\partial RSS}{\partial \mathbf{w}} \stackrel{!}{=} \mathbf{0}.$$

We need some properties of vector derivatives:

$$\partial(A\mathbf{x})/\partial \mathbf{x} = A^t$$

$$\partial(\mathbf{x}^t A)/\partial \mathbf{x} = A$$

$$\partial(\mathbf{x}^t A \mathbf{x})/\partial \mathbf{x} = A\mathbf{x} + A^t \mathbf{x} \quad (\text{if } A \text{ is square})$$

Normal Equations

$$\begin{aligned}\frac{\partial RSS}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} [\mathbf{y}^t \mathbf{y} - 2\mathbf{y}^t X \mathbf{w} + \mathbf{w}^t X^t X \mathbf{w}] \\ &= -2X^t \mathbf{y} + [X^t X \mathbf{w} + (X^t X)^t \mathbf{w}] \\ &= -2X^t \mathbf{y} + 2X^t X \mathbf{w} = \mathbf{0}\end{aligned}$$

Normal equations: $X^t X \mathbf{w} = X^t \mathbf{y}$.

Could solve this system. **But:** All solution methods based on normal equations are **inherently susceptible to roundoff errors**:

$$\begin{aligned}k(X) &= \sigma_{\max}/\sigma_{\min}, \text{ where } X^t X \mathbf{v}_i = \sigma_i^2 \mathbf{v}_i \\ k(X^t X) &= \mu_{\max}/\mu_{\min}, \text{ where } X^t X X^t X \mathbf{v}_i = \mu_i^2 \mathbf{v}_i \\ X^t X X^t X \mathbf{v}_i &= X^t X \sigma_i^2 \mathbf{v}_i = \sigma_i^4 \mathbf{v}_i \Rightarrow \mu_i = \sigma_i^2 \\ \Rightarrow k(X^t X) &= k^2(X),\end{aligned}$$

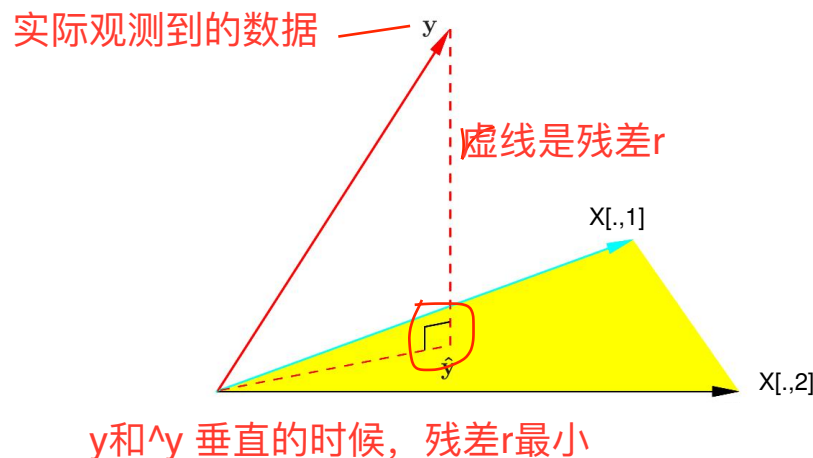
The algebraic approach will avoid this problem!

From Calculus to Algebra

$$\frac{\partial RSS(\mathbf{w})}{\partial \mathbf{w}} = -2X^t \mathbf{y} + 2X^t X \mathbf{w} \stackrel{!}{=} \mathbf{0}$$

$$\Rightarrow X^t(\mathbf{y} - X\hat{\mathbf{w}}) = X^t \mathbf{r} = \mathbf{0} \quad \Rightarrow \mathbf{r} \in N(X^t).$$

- Every $X\mathbf{w}$ is in column space $C(X)$,
residual $\mathbf{r}$ is in the orthogonal complement $N(X^t)$ (left nullspace).
- Let $\hat{\mathbf{y}}$ be the **orthogonal projection** of $\mathbf{y}$ on $C(X)$
 $\rightsquigarrow \mathbf{y}$ can be split into $\hat{\mathbf{y}} \in C(X) + \mathbf{r} \in N(X^t)$.



Adapted from Fig. 3.2 in (Hastie, Tibshirani, Friedman)

Algebraic interpretation

- $\mathbf{y} = \hat{\mathbf{y}} \in C(X) + \mathbf{r} \in N(X^t) \rightsquigarrow$ Consider over-determined systems

$$X\mathbf{w} = \mathbf{y} = \hat{\mathbf{y}} + \mathbf{r} \text{ (solution impossible, if } \mathbf{r} \neq \mathbf{0}\text{)}$$

$$X\hat{\mathbf{w}} = \hat{\mathbf{y}} \text{ (solvable, since } \hat{\mathbf{y}} \in C(X)\text{!)}$$

- **The solution $\hat{\mathbf{w}}$ of $X\mathbf{w} = \hat{\mathbf{y}}$ makes the error as small as possible:**

$$\|X\mathbf{w} - \mathbf{y}\|^2 = \|X\mathbf{w} - (\hat{\mathbf{y}} + \mathbf{r})\|^2 = \|X\mathbf{w} - \hat{\mathbf{y}}\|^2 + \|\mathbf{r}\|^2$$

Reduce $\|X\mathbf{w} - \hat{\mathbf{y}}\|^2$ to zero by solving $X\hat{\mathbf{w}} = \hat{\mathbf{y}}$ and choosing $\mathbf{w} = \hat{\mathbf{w}}$.
Remaining error $\|\mathbf{r}\|^2$ cannot be avoided, since $\mathbf{r} \in N(X^t)$.

$$X^t X \hat{\mathbf{w}} = X^t \hat{\mathbf{y}} = X^t \mathbf{y} \quad \Rightarrow \quad \hat{\mathbf{w}} = (X^t X)^{-1} X^t \mathbf{y} \text{ (if } X^t X \text{ invertible).}$$

- The fitted values at the sample points are $\hat{\mathbf{y}} = X\hat{\mathbf{w}} = X(X^t X)^{-1} X^t \mathbf{y}$.
- $H = X(X^t X)^{-1} X^t$ is called **hat matrix** (puts a “hat” on $\mathbf{y} \rightsquigarrow \hat{\mathbf{y}}$).

Algebraic interpretation

- Left nullspace $N(X^t)$ is orthogonal complement of column space $C(X)$.
- H is **orthogonal projection** on $C(X)$:

$$HX = X(X^tX)^{-1}X^tX = X, \quad HN(X^t) = 0.$$

- $M = I - H$ is **orthogonal projection** on **nullspace** of X^t :

$$MX = (I - H)X = X - X = 0, \quad MN(X^t) = M.$$

- H and M are symmetric ($H^t = H$) and idempotent ($MM = M$)

The algebra of Least Squares:

H creates fitted values: $\hat{\mathbf{y}} = H\mathbf{y} \rightsquigarrow \hat{\mathbf{y}} \in C(X)$

M creates residuals: $\mathbf{r} = M\mathbf{y} \rightsquigarrow \mathbf{r} \in N(X^t)$

Algebraic interpretation

$X^t X$ is invertible iff X has linearly independent columns.

Why? $X^t X$ has the same nullspace as X :

(i) If $\mathbf{a} \in N(X)$, then $X\mathbf{a} = \mathbf{0} \Rightarrow X^t X\mathbf{a} = \mathbf{0} \rightsquigarrow \mathbf{a} \in N(X^t X)$.

(ii) If $\mathbf{a} \in N(X^t X)$, then $\mathbf{a}^t X^t X\mathbf{a} = 0 \Leftrightarrow \|X\mathbf{a}\|^2 = 0$,

so $X\mathbf{a}$ has length zero $\Rightarrow X\mathbf{a} = \mathbf{0}$.

Thus, every vector in one nullspace is also in the other one.

So if $N(X) = \{\mathbf{0}\}$, then $X^t X \in \mathbb{R}^{d \times d}$ has full rank d .

When X has independent columns, $X^t X$ is positive definite.

Why? $X^t X$ is clearly symmetric and invertible.

To show: All eigenvalues > 0

$$X^t X \mathbf{v} = \lambda \mathbf{v} \rightsquigarrow \mathbf{v}^t X^t X \mathbf{v} = \lambda \mathbf{v}^t \mathbf{v} \rightsquigarrow \lambda = \frac{\|X\mathbf{v}\|^2}{\|\mathbf{v}\|^2} > 0.$$

SVD for Least-Squares

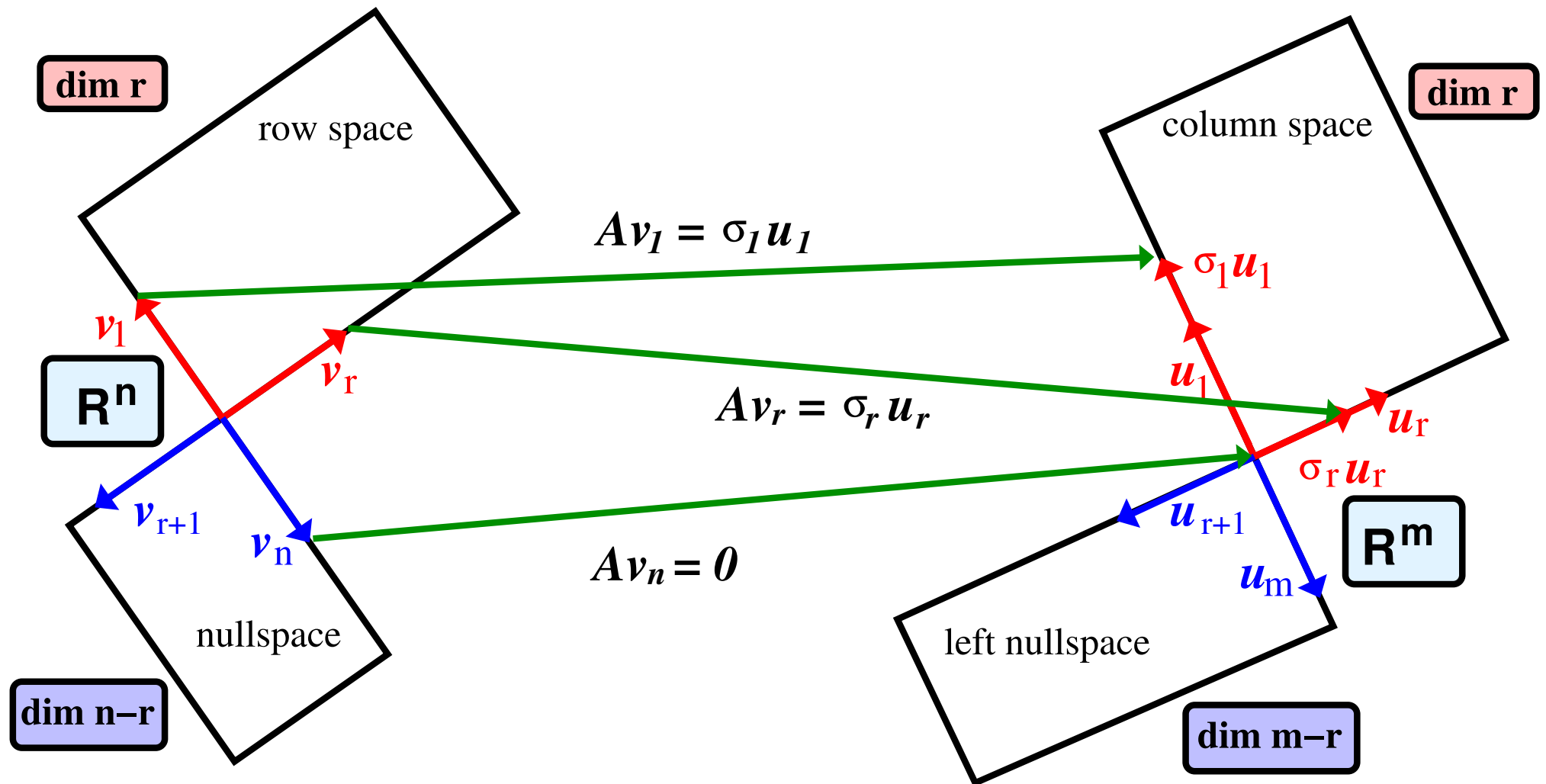
- Goal: Avoid numerical problems for normal equations:
 $X^t X \mathbf{w} = X^t \mathbf{y}, \quad k(X^t X) = k^2(X).$
- Idea: Apply the **SVD** directly to $X_{n \times d}$.
- The **squared norm of the residual** is

$$\begin{aligned} RSS &= \|\mathbf{r}\|^2 = \|X\mathbf{w} - \mathbf{y}\|^2 \\ &= \|USV^t\mathbf{w} - \mathbf{y}\|^2 \\ &= \|U(SV^t\mathbf{w} - U^t\mathbf{y})\|^2 \\ &= \|SV^t\mathbf{w} - U^t\mathbf{y}\|^2 \end{aligned}$$

Last equation: U is orthonormal $\leadsto \|U\mathbf{a}\|^2 = \mathbf{a}^t U^t U \mathbf{a} = \mathbf{a}^t \mathbf{a} = \|\mathbf{a}\|^2$.

- Minimizing RSS is **equivalent** to minimizing $\|S\mathbf{z} - \mathbf{c}\|^2$ where $\mathbf{z} = V^t\mathbf{w}$ and $\mathbf{c} = U^t\mathbf{y}$.

SVD and bases for the 4 subspaces



SVD and LS

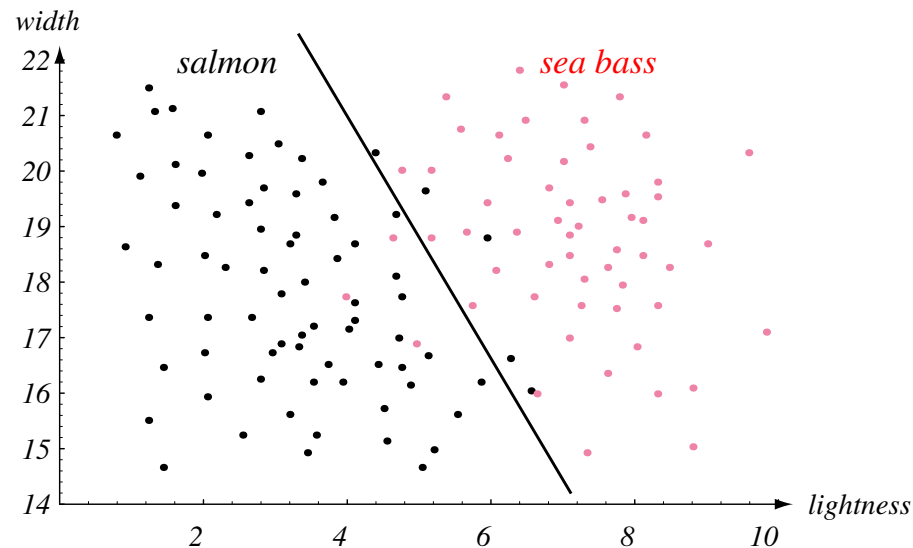
- $\|\mathbf{r}\|^2 = \|S\mathbf{z} - \mathbf{c}\|^2$ written in blocks:

$$\left\| \begin{bmatrix} \sigma_1 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 \\ 0 & 0 & \dots & \sigma_d \\ \hline 0 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_d \end{bmatrix} - \begin{bmatrix} c_1 \\ \vdots \\ c_d \\ c_{d+1} \\ \vdots \\ c_n \end{bmatrix} \right\|^2$$

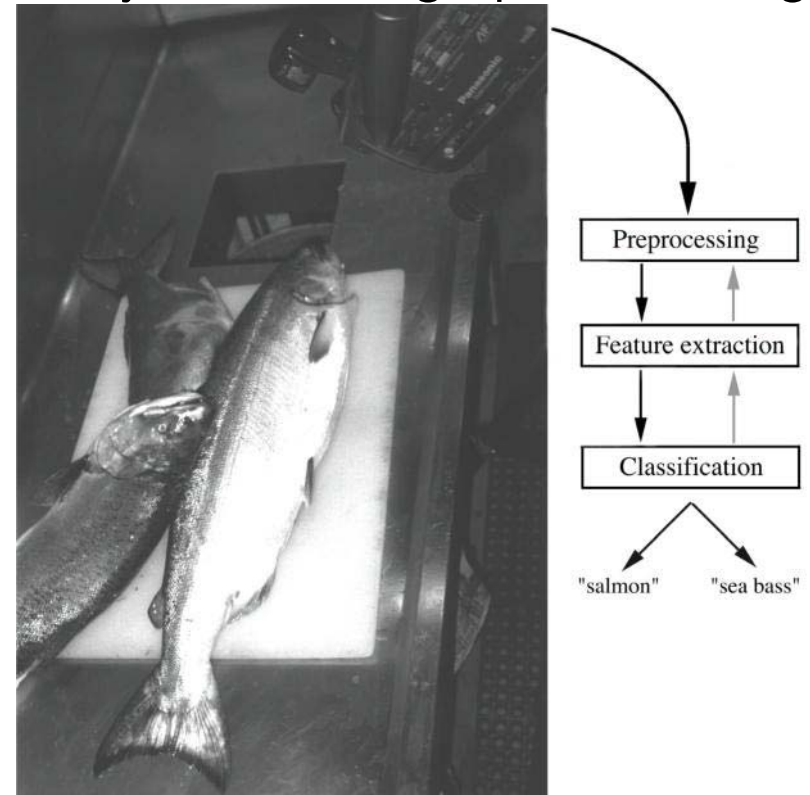
- To choose $\mathbf{z}$ so that $\|\mathbf{r}\|^2$ is minimal requires $z_i = c_i/\sigma_i, i = 1, \dots, d$
 $\leadsto r_1 = r_2 = \dots = r_d = 0$.
- Unavoidable error: $RSS = \|\mathbf{r}\|^2 = c_{d+1}^2 + c_{d+2}^2 + \dots + c_n^2$.
- For very small singular values, use zeroing. RSS will increase:
 One additional term (usually small): $RSS' = c_d^2 + c_{d+1}^2 + c_{d+2}^2 + \dots + c_n^2$,
 but often significantly better precision (reduced condition number).

Classification

Classification: Find **class boundaries** based on training data $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$. Use boundaries to classify new items $\mathbf{x}^*$. Here, y_i is a discrete class indicator (or “label”). Example: Fish-packing plant wants to automate the process of sorting fish on conveyor belt using optical sensing.

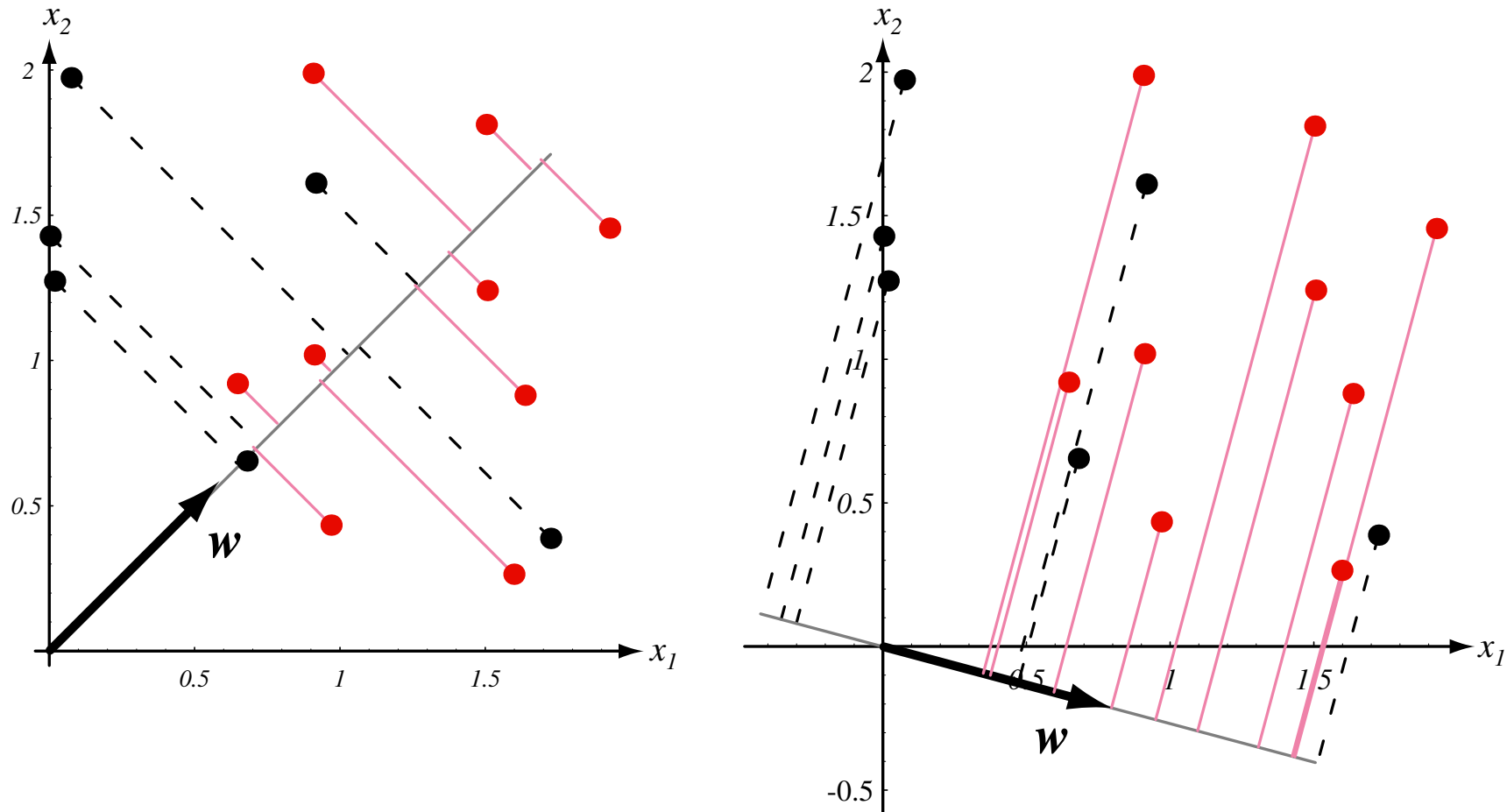


(Duda, Hart, Stork, 2001)



(Duda, Hart, Stork, 2001)

Linear Discriminant Analysis (Ronald Fisher, 1936)



(Duda, Hart, Stork, 2001)

Main Idea: Simplify the problem by projecting down to a 1-dim subspace.
Question: How should we select the **projection vector**, which optimally discriminates between the different classes?

Separation Criterion

- Let $\mathbf{m}_j$ an estimate of the class means $\boldsymbol{\mu}_j$:

$$\mathbf{m}_y = \frac{1}{n_y} \sum_{\mathbf{x} \in \text{class } y} \mathbf{x}, \quad n_y = \#(\text{objects in class } y).$$

- Projected samples: $\mathbf{x}'_i = \mathbf{w}^t \mathbf{x}_i$, $i = 1, 2, \dots, n$. Projected means:

$$\tilde{\mathbf{m}}_y = \frac{1}{n_y} \sum_{\mathbf{x} \in \text{class } y} \mathbf{w}^t \mathbf{x} = \mathbf{w}^t \mathbf{m}_y.$$

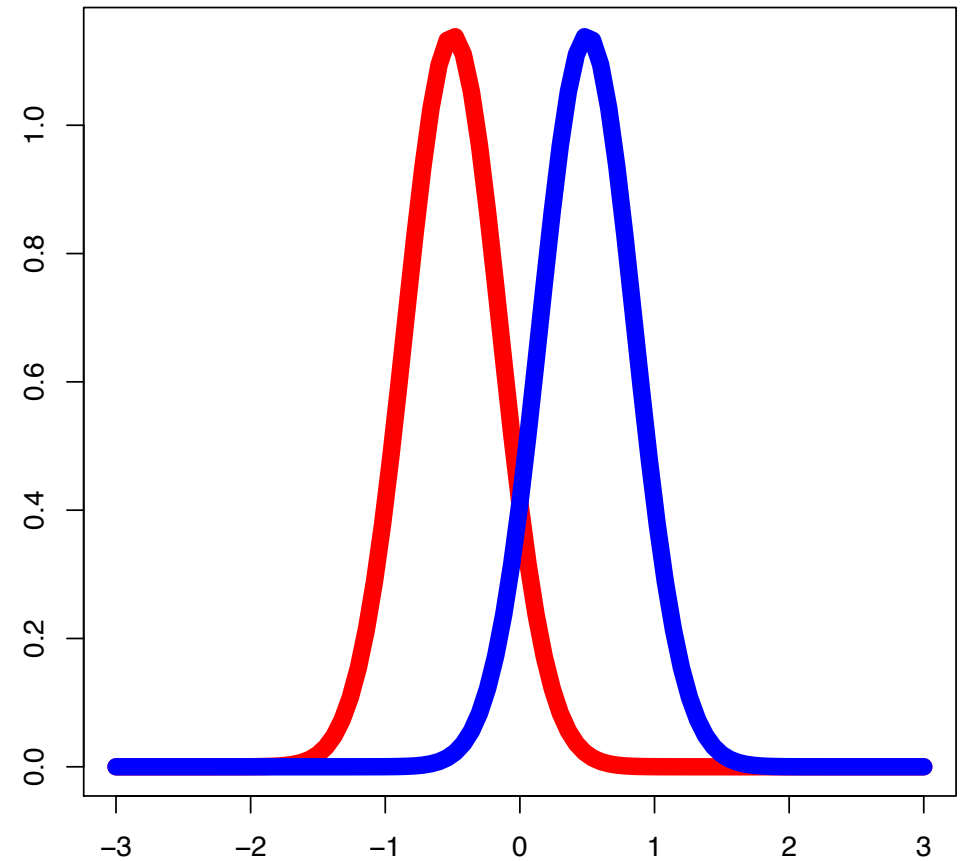
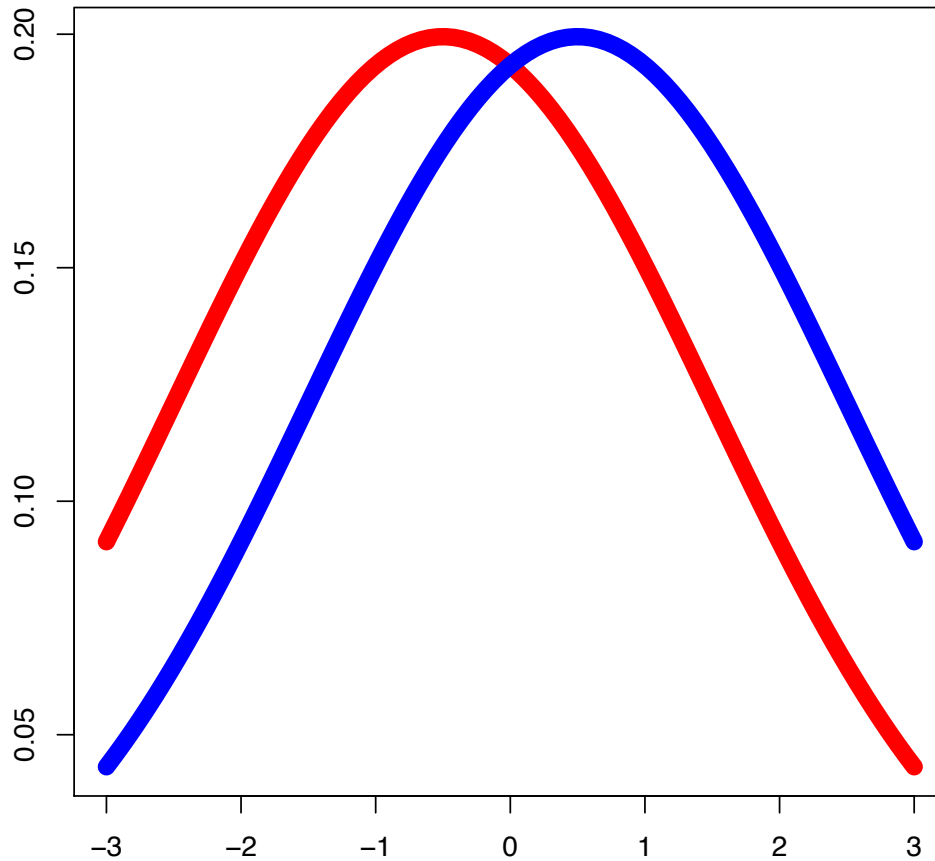
- First part of separation criterion (two-class case):

$$\max_{\mathbf{w}} [\mathbf{w}^t (\mathbf{m}_1 - \mathbf{m}_2)]^2 = \max_{\mathbf{w}} [\tilde{\mathbf{m}}_1 - \tilde{\mathbf{m}}_2]^2.$$

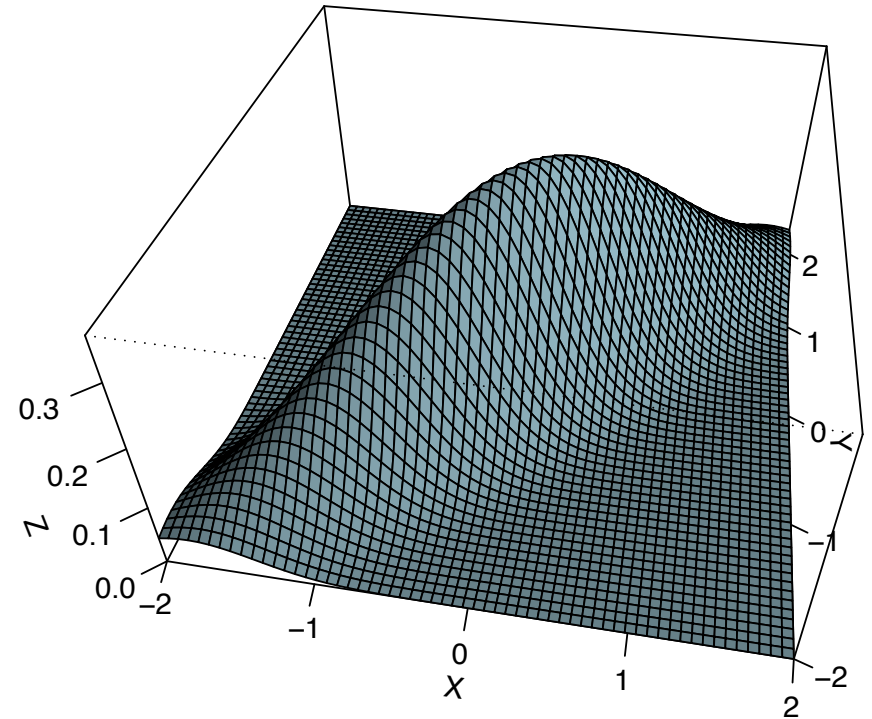
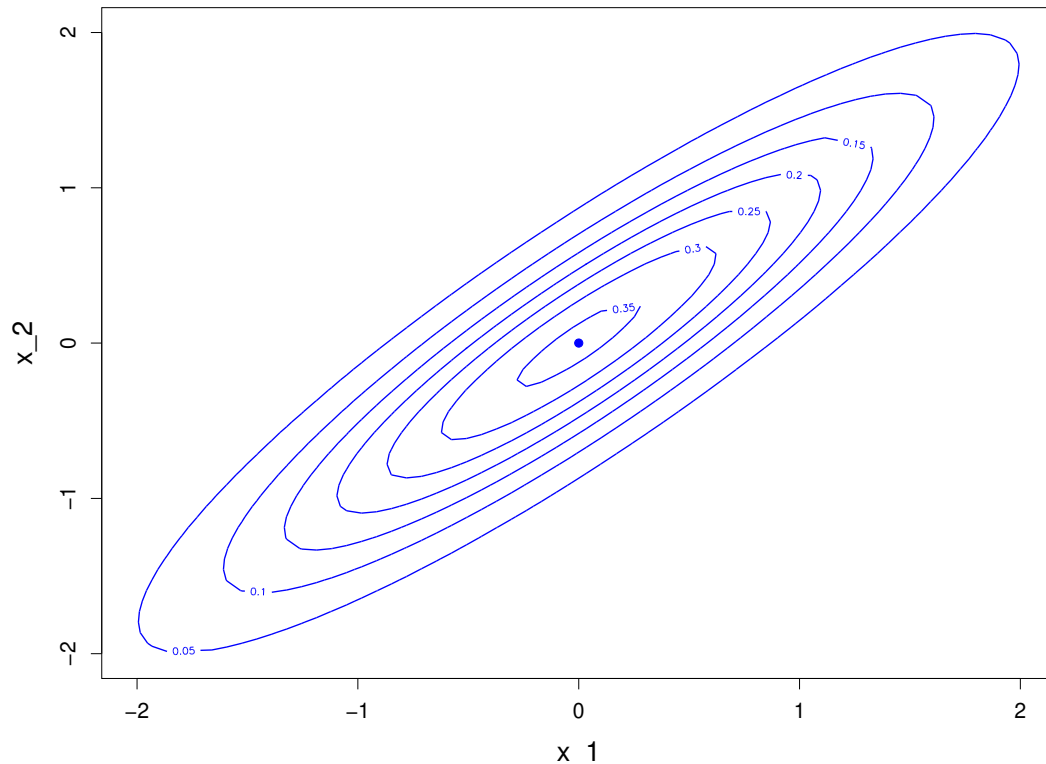
- There might still be considerable overlap...
 $\rightsquigarrow$ should also consider the **scatter** or **variance**.

Separation Criterion

Two Gaussians with the same mean distance, but different variances:



Excursion: The multivariate Gaussian distribution



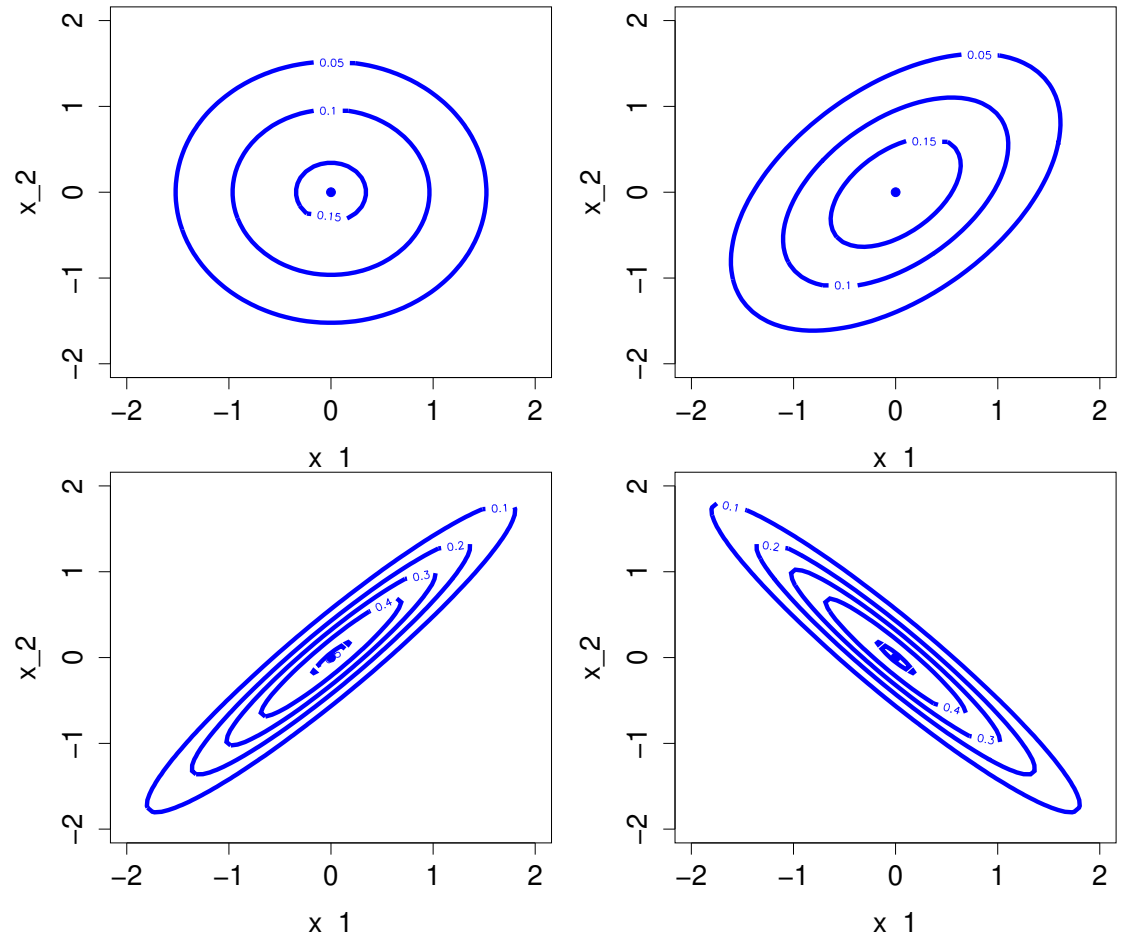
Probability density function:

$$p(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{2\pi|\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^t \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

Excursion: The multivariate Gaussian distribution

Covariance

(also written “co-variance”)
is a measure of how much
two random variables vary together. Can be positive, zero, or negative.



Sample covariance matrix $\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^t$,
with sample mean $\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \mathbf{m}$. If $\mathbf{m} = \mathbf{0} \rightsquigarrow \hat{\Sigma} = \frac{1}{n} \mathbf{X}^t \mathbf{X}$.

Separation Criterion

- Assume both classes are Gaussians with the same covariance matrix. Let Σ_W be an estimate of this **“within class” covariance matrix**:

$$\Sigma_y = \frac{1}{n_y} \sum_{\mathbf{x} \in \text{class } y} (\mathbf{x} - \mathbf{m}_y)(\mathbf{x} - \mathbf{m}_y)^t,$$
$$\Sigma_W = 0.5(\Sigma_1 + \Sigma_2).$$

- Variance of projected data:

$$\begin{aligned}\tilde{\Sigma}_y &= \frac{1}{n_y} \sum_{\mathbf{x} \in \text{class } y} (\mathbf{w}^t \mathbf{x} - \tilde{m}_y)(\mathbf{w}^t \mathbf{x} - \tilde{m}_y)^t \\ &= \frac{1}{n_y} \sum_{\mathbf{x} \in \text{class } y} \mathbf{w}^t (\mathbf{x} - \mathbf{m}_y)(\mathbf{x} - \mathbf{m}_y)^t \mathbf{w} = \mathbf{w}^t \Sigma_y \mathbf{w} \\ \tilde{\Sigma}_W &= 0.5(\tilde{\Sigma}_1 + \tilde{\Sigma}_2) = \mathbf{w}^t \Sigma_W \mathbf{w} \in \mathbb{R}_+\end{aligned}$$

- Strategy: $\Delta_{\tilde{\mathbf{m}}}^2 = (\tilde{m}_1 - \tilde{m}_2)^2$ should be large, $\tilde{\Sigma}_W$ small.

Separation Criterion

$$J(\mathbf{w}) = \frac{\Delta_{\tilde{m}}^2}{\tilde{\Sigma}_W} = \frac{\mathbf{w}^t \overbrace{(\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^t}^{=: \Sigma_B} \mathbf{w}}{\mathbf{w}^t \Sigma_W \mathbf{w}}.$$

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}} J(\mathbf{w}) &= \frac{\partial}{\partial \mathbf{w}} \frac{\mathbf{w}^t \Sigma_B \mathbf{w}}{\mathbf{w}^t \Sigma_W \mathbf{w}} \stackrel{!}{=} 0 \\ &= -\frac{\mathbf{w}^t \Sigma_B \mathbf{w}}{(\mathbf{w}^t \Sigma_W \mathbf{w})^2} 2 \Sigma_W \mathbf{w} + \frac{1}{\mathbf{w}^t \Sigma_W \mathbf{w}} 2 \Sigma_B \mathbf{w} \\ &\Rightarrow \frac{\mathbf{w}^t \Sigma_B \mathbf{w}}{\mathbf{w}^t \Sigma_W \mathbf{w}} (-\Sigma_W \mathbf{w}) + \Sigma_B \mathbf{w} = 0 \\ &\Rightarrow \Sigma_B \mathbf{w} = \frac{\mathbf{w}^t \Sigma_B \mathbf{w}}{\mathbf{w}^t \Sigma_W \mathbf{w}} \Sigma_W \mathbf{w} =: \lambda \Sigma_W \mathbf{w} \end{aligned}$$

Separation Criterion

- Let Σ_W be non-singular:

$$\underbrace{\left[\Sigma_W^{-1} \quad \Sigma_B \right] \mathbf{w}}_{\Delta \mathbf{m} \Delta \mathbf{m}^t \mathbf{w} \propto \Delta \mathbf{m}} = \lambda \mathbf{w}, \quad \text{with} \quad \lambda = \frac{\mathbf{w}^t \Sigma_B \mathbf{w}}{\mathbf{w}^t \Sigma_W \mathbf{w}} = J(\mathbf{w}).$$

- Thus, $\mathbf{w}$ is an eigenvector of $\Sigma_W^{-1} \Sigma_B$, the associated eigenvalue is the objective function! **Maximum: eigenvector with largest eigenvalue.**
- Unscaled Solution: $\hat{\mathbf{w}} = \Sigma_W^{-1} \Delta \mathbf{m} = \Sigma_W^{-1} (\mathbf{m}_1 - \mathbf{m}_2).$
- This is the solution of the linear system $\Sigma_W \mathbf{w} = \mathbf{m}_1 - \mathbf{m}_2.$
- Σ_W is a covariance matrix $\rightsquigarrow$ there is an underlying data matrix A such that $\Sigma_W \propto A^t A \rightsquigarrow$ potential numerical problems: squared condition number compared to $A...$

Discriminant analysis and least squares

Theorem: The LDA vector $\hat{\mathbf{w}}^{\text{LDA}} = \Sigma_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$ coincides with the solution of the LS problem $\hat{\mathbf{w}}^{\text{LS}} = \arg \min_{\mathbf{w}} \|X\mathbf{w} - \mathbf{y}\|^2$ if

n_1 = # samples in class 1

n_2 = # samples in class 2

$$X = \begin{bmatrix} - & \mathbf{x}_1^t & - \\ - & \mathbf{x}_2^t & - \\ & \vdots & \\ - & \mathbf{x}_n^t & - \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix},$$

with $\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \mathbf{m} = \mathbf{0}$ (i.e. origin in sample mean),

$$y_i = \begin{cases} +1/n_1, & \text{if } \mathbf{x}_i \text{ in class 1} \\ -1/n_2, & \text{else.} \end{cases} \quad \Rightarrow \quad \sum_{i=1}^n y_i = 0.$$

Discriminant analysis and least squares (cont'd)

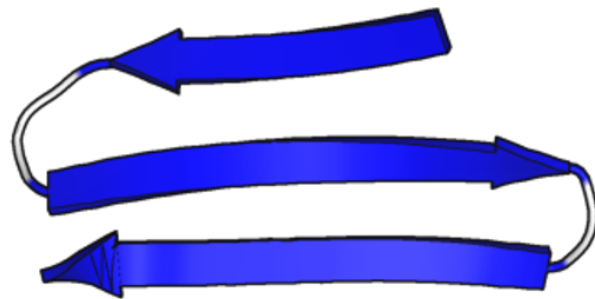
- “Within” covariance $\Sigma_W \propto \sum_{\mathbf{x} \in \text{class } y} (\mathbf{x} - \mathbf{m}_y)(\mathbf{x} - \mathbf{m}_y)^t$.
- “Between” covariance $\Sigma_B \propto (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^t$
- The sum of both is the “total covariance” $\Sigma_B + \Sigma_W = \Sigma_T$
 $\Sigma_T \propto \sum_i \mathbf{x}_i \mathbf{x}_i^t = X^t X$.
- We know that $\mathbf{w}^{\text{LDA}} \propto \Sigma_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2) \rightsquigarrow \Sigma_W \mathbf{w}^{\text{LDA}} \propto (\mathbf{m}_1 - \mathbf{m}_2)$.
- Now $\Sigma_B \mathbf{w}^{\text{LDA}} = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^t \mathbf{w}^{\text{LDA}} \rightsquigarrow \Sigma_B \mathbf{w}^{\text{LDA}} \propto (\mathbf{m}_1 - \mathbf{m}_2)$.
- $\Sigma_T \mathbf{w}^{\text{LDA}} = (\Sigma_B + \Sigma_W) \mathbf{w}^{\text{LDA}} \rightsquigarrow \Sigma_T \mathbf{w}^{\text{LDA}} \propto (\mathbf{m}_1 - \mathbf{m}_2)$.
- With $X^t X = \Sigma_T$, $X^t \mathbf{y} = \mathbf{m}_1 - \mathbf{m}_2$, we arrive at
 $\mathbf{w}^{\text{LDA}} \propto \Sigma_T^{-1}(\mathbf{m}_1 - \mathbf{m}_2) = \Sigma_T^{-1} X^t \mathbf{y} \propto (X^t X)^{-1} X^t \mathbf{y} = \mathbf{w}^{\text{LS}}$.

Chapter 2

Least squares problems

Application Example: Secondary Structure Prediction in Proteins

Secondary



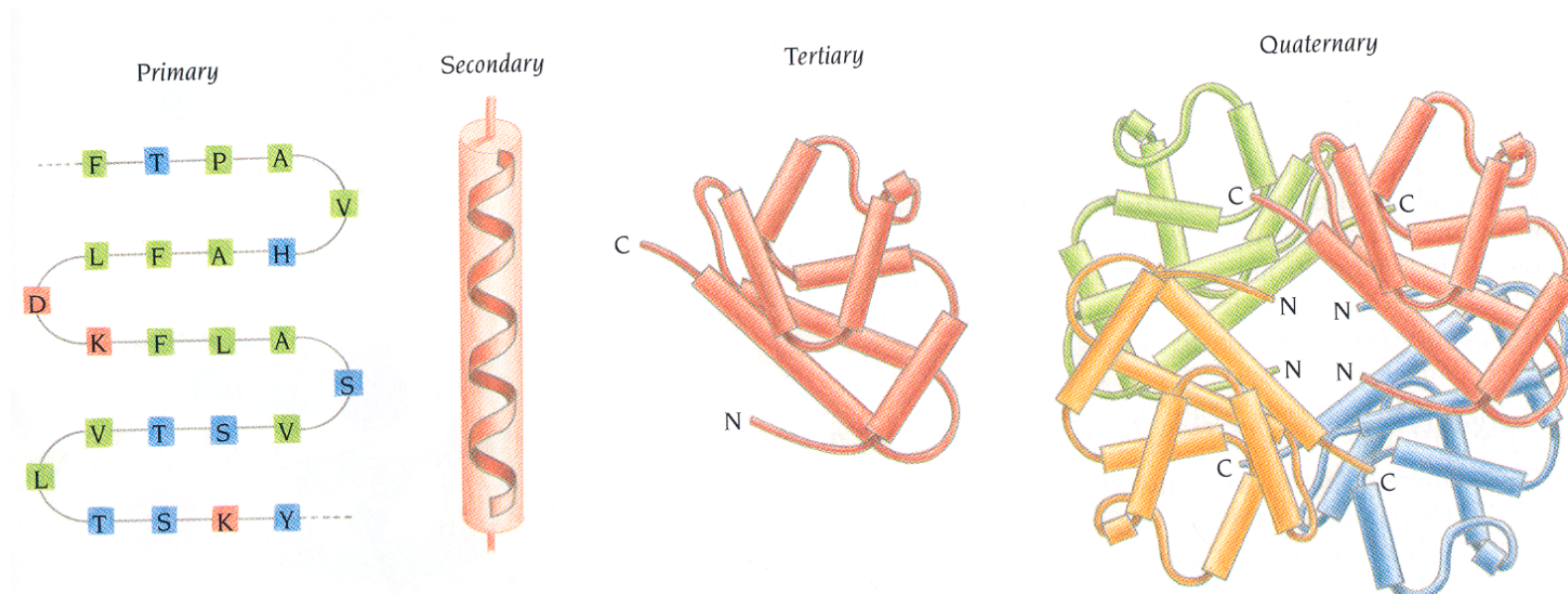
β -Sheet (3 strands)



α -helix

Proteins

- To a large extent, **cells are made of proteins.**
- Proteins are large biomolecules: **linear polymers of amino acids.**
Average size ≈ 200 amino acids, can be over 1000.
- **Complex structure** with four hierarchical levels.

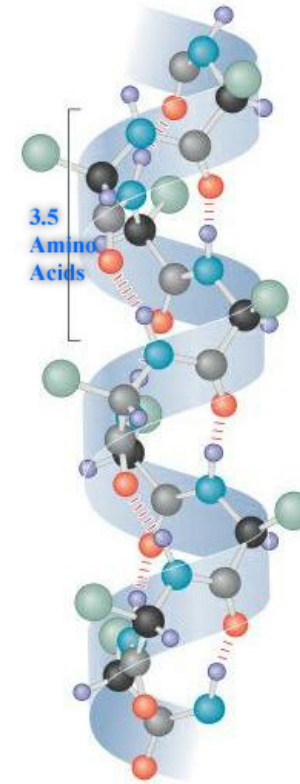
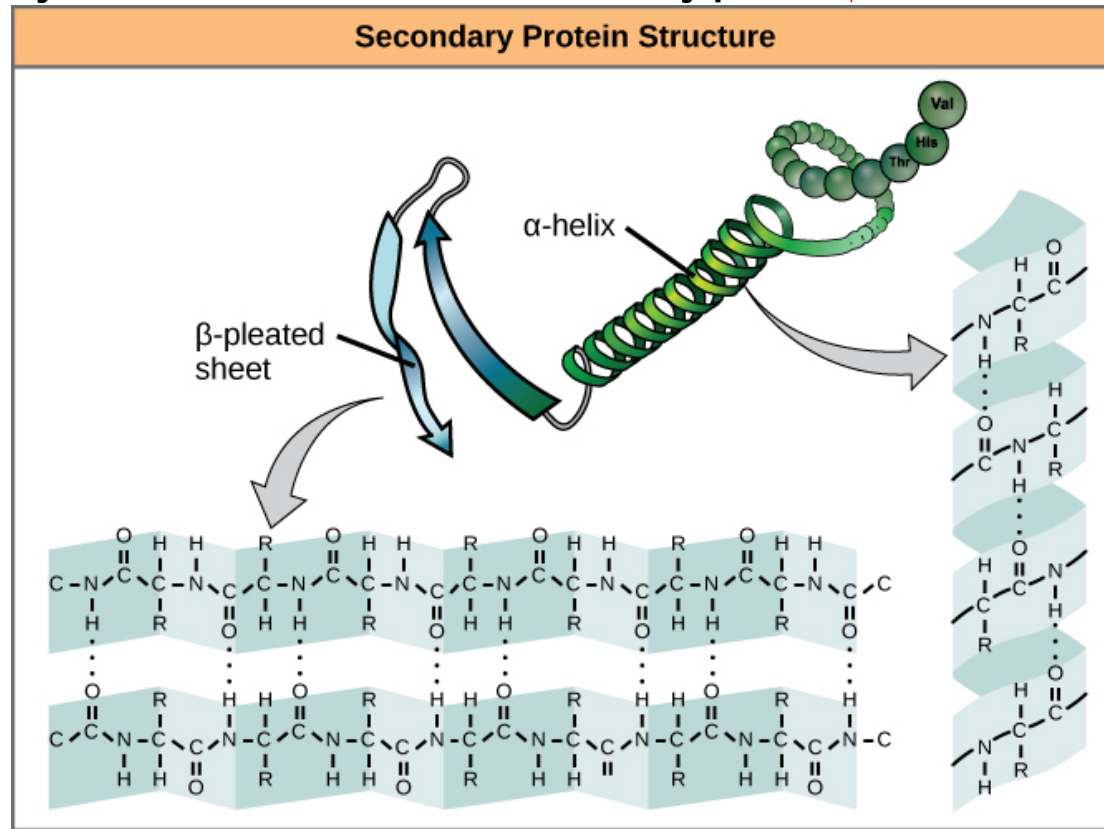


Durbin et al., Cambridge University Press

Structure determined by the **primary sequence** and their **physico-chemical interactions** in the medium. **Structure determines functionality.**

Secondary Structure

Secondary structure: two main types: β -sheet and α -helix



The School of Biomedical Sciences Wiki

Short range interactions in the AA chain are important for the secondary structure:
 α -helix performs a 100° turn *per amino acid* $\leadsto$ full turn after 3.6 AAs.
Formation of a helix mainly depends on interactions in a **4 AA** window.

Example: Cytochrome C2 Precursor

Secondary structure (h=helix)

amino acid sequence

hhhhhhhhhh

MKKGFLAAGVFAAVAFASGAALAEGDAAAGEKVSKKCLACHTFDQGGANKVGPNLFGVFE

hhhhhhhh hhhhhhhh hhhhhhhh

NTAAHKDDYAYSESYTEMKAKGLTWTEANLAAYVKDPKAFVLEKSGDPKAKSKMTFKLTK

hhhhhhhhhhhh

DDEIENVIAYLKTLK



Given: Examples of known helices and non-helices in several proteins

~> **training set**

Goal: Predict, mathematically, the existence and position of α -helices in **new proteins.**

Classification of Secondary Structure

Idea: Use a **sliding window** to cut the AA chain into pieces. 4 AAs are enough to capture one full turn $\rightsquigarrow$ choose window of size 5.

Decision Problem: Find function $f(\dots)$ that predicts for each substring in a window the structure:

$$f(\text{AADTG}) = \begin{cases} \text{"Yes"}, & \text{if the central AA belongs to an } \alpha\text{-helix,} \\ \text{"No"}, & \text{otherwise} \end{cases}$$

Problem: How should we numerically encode a string like AADTG?

Simple encoding scheme: **Count the number of occurrences of each AA in the window.** First order approximation, neglects AA's position within the window.

Example

...RAADTGGSDP...

...xx~~x~~h~~h~~h~~h~~h~~x~~...

...x~~x~~xh~~h~~h~~h~~h~~x~~...

...x~~x~~xh~~h~~h~~h~~h~~x~~...

(black $\hat{=}$ structure info about central AA; green $\hat{=}$ know secondary structure; red $\hat{=}$ sliding window)

A	C	D	...	G	...	R	S	T	...	Y	Label
2	0	1	0	0	0	1	0	1	0	0	"No"
2	0	1	0	1	0	0	0	1	0	0	"Yes"
1	0	1	0	2	0	0	0	1	0	0	"Yes"
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

This is a binary classification problem

⇒ use **Linear Discriminant Analysis**

Discriminant Analysis

Consider $X_{n \times d}$, with $n = \#(\text{windows})$ and $d = \#(\text{AAs}) = 20(\text{or } 21)$, and the n -vector of class indicators $\mathbf{y}$

$$X = \begin{bmatrix} 2 & 0 & 1 & \dots & 0 & \dots \\ 2 & 0 & 1 & \dots & 1 & \dots \\ 1 & 0 & 1 & \dots & 2 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} = \begin{bmatrix} - & \mathbf{x}_1^t & - \\ - & \mathbf{x}_2^t & - \\ & \vdots & \\ - & \mathbf{x}_n^t & - \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} \text{"No"} \\ \text{"Yes"} \\ \text{"Yes"} \\ \vdots \end{bmatrix}$$

For the binary class indicators, we use some numerical encoding scheme.

Interpretation with basis functions:

$\mathbf{x}$ = sequence of characters from alphabet $\mathcal{A}$

$g_i(\mathbf{x})$ = $\#(\text{occurences of letter } i \text{ in sequence})$

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^t \mathbf{g} = \sum_{i \in \text{characters}} w_i g_i(\mathbf{x})$$

Discriminant analysis and least squares

Recall: The LDA vector $\hat{\mathbf{w}}^{\text{LDA}} = \Sigma_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$ coincides with the solution of the LS problem $\hat{\mathbf{w}}^{\text{LS}} = \arg \min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$ if

n_1 = # samples in class 1

n_2 = # samples in class 2

$$\mathbf{X} = \begin{bmatrix} - & \mathbf{x}_1^t & - \\ - & \mathbf{x}_2^t & - \\ & \vdots & \\ - & \mathbf{x}_n^t & - \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix},$$

with $\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \mathbf{m} = \mathbf{0}$ (i.e. origin in sample mean),

$$y_i = \begin{cases} +1/n_1, & \text{if } \mathbf{x}_i \text{ in class 1} \\ -1/n_2, & \text{else.} \end{cases} \quad \Rightarrow \quad \sum_{i=1}^n y_i = 0$$

Singular Value Decomposition (SVD)

Recall: SVD for nonsquare matrix $X \in \mathbb{R}^{n \times d}$: $X = USV^t$.

Residual sum of squares:

$$RSS = \|\mathbf{r}\|^2 = \|X\mathbf{w} - \mathbf{y}\|^2 = \|USV^t\mathbf{w} - \mathbf{y}\|^2 = \|\underbrace{SV^t\mathbf{w}}_{\mathbf{z}} - \underbrace{U^t\mathbf{y}}_{\mathbf{c}}\|^2$$

Minimizing $\|\mathbf{r}\|^2$ is equivalent to minimizing $\|S\mathbf{z} - \mathbf{c}\|^2$:

$$\text{minimize } \|\mathbf{r}\|^2 = \left\| \begin{bmatrix} \sigma_1 & & 0 \\ & \ddots & \\ 0 & & \sigma_d \\ \hline 0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 0 \end{bmatrix} \cdot \begin{bmatrix} z_1 \\ \vdots \\ z_d \end{bmatrix} - \begin{bmatrix} c_1 \\ \vdots \\ c_d \\ c_{d+1} \\ \vdots \\ c_n \end{bmatrix} \right\|^2$$

We now choose z_k so that $\|\mathbf{r}\|^2$ is minimal, i.e., for $\sigma_k > 0$:

$$z_k = \frac{c_k}{\sigma_k}$$

Iterative Algorithm

In our problem we have $d = 20$ (or 21) and $n > 10000$.

Goal: Use only $X^t X \in \mathbb{R}^{d \times d}$ and $X^t \mathbf{y} \in \mathbb{R}^d$.

Initialize $X^t X = 0$ (zero matrix), $X^t \mathbf{y} = \mathbf{0}$. **Update:** for $j = 1$ to n :

$$X^t X + \mathbf{x}_j \mathbf{x}_j^t \longrightarrow X^t X$$

$$X^t \mathbf{y} + \mathbf{x}_j y_j \longrightarrow X^t \mathbf{y}$$

The first update procedure is correct, since

$$\begin{aligned} (X^t X)_{ik} &= \sum_{j=1}^n x_{ji} x_{jk} \\ \Rightarrow X^t X &= \sum_{j=1}^n \begin{bmatrix} x_{j1} \\ x_{j2} \\ \vdots \\ x_{jd} \end{bmatrix} \cdot [x_{j1}, x_{j2}, \dots, x_{jd}] = \sum_{j=1}^n \mathbf{x}_j \mathbf{x}_j^t \end{aligned}$$

Iterative Algorithm

A similar calculation yields the other equation:

$$(X^t \mathbf{y})_i = \sum_j x_{ji} y_j \Rightarrow X^t \mathbf{y} = \sum_j \begin{bmatrix} x_{j1} \\ x_{j2} \\ \vdots \\ x_{jd} \end{bmatrix} \cdot y_j = \sum_{j=1}^n \mathbf{x}_j y_j$$

One remaining problem: In LDA we assumed that X was centered, i.e. the column sums are all zero. Compute the column sums as:

$$\mathbf{1}^t X = [1, 1, \dots, 1] \begin{bmatrix} - & \mathbf{x}_1^t & - \\ - & \mathbf{x}_2^t & - \\ & \vdots & \\ - & \mathbf{x}_n^t & - \end{bmatrix} = n \cdot [m_1, m_2, \dots, m_d] = n \cdot \mathbf{m}^t$$

$$\rightsquigarrow \text{“centered” } X_c = X - \mathbf{1} \mathbf{m}^t = X - \frac{1}{n} \mathbf{1} \mathbf{1}^t X$$

Centering

$$\begin{aligned}X_c &= X - \mathbf{1}m^t = X - \frac{1}{n}\mathbf{1}\mathbf{1}^t X \\X_c^t X_c &= X^t X + \frac{1}{n^2} X^t \mathbf{1} \underbrace{\mathbf{1}^t \mathbf{1}}_{=n} \mathbf{1}^t X - \frac{1}{n} X^t \mathbf{1} \mathbf{1}^t X - \frac{1}{n} X^t \mathbf{1} \mathbf{1}^t X \\&= X^t X - \frac{1}{n} X^t \mathbf{1} \mathbf{1}^t X \\&= X^t X - n \cdot m m^t\end{aligned}$$

Iteratively update the vector $n \cdot m$ for every x_i corresponding to a new window position: **Initialize** $n \cdot m = 0$ **and update** $n \cdot m \leftarrow n \cdot m + x_i$

What about $X^t y$? We should have used

$$X_c^t y = (X - \mathbf{1}m^t)^t y = (X^t - m \mathbf{1}^t) y = X^t y - m \mathbf{1}^t y$$

But by construction, y is orthogonal to $\mathbf{1} \rightsquigarrow \mathbf{1}^t y = 0$,
so nothing needs to be done!

Iterative Algorithm

Goal: Solution which only requires $X_c^t X_c \in \mathbb{R}^{d \times d}$ and $X_c^t \mathbf{y} \in \mathbb{R}^d$ alone (and does not use X_c or $\mathbf{y}$ explicitly).

We need:

- The matrix V (for computing $\hat{\mathbf{w}} = V \mathbf{z}$)

Solution: columns of V are the eigenvectors of $X_c^t X_c$,
corresponding eigenvalues are λ_i , $i = 1, \dots, n \Rightarrow \sigma_i^2 = \lambda_i$

- For the nonzero SVs, we need $z_i = (U^t \mathbf{y})_i / \sigma_i = \sigma_i (U^t \mathbf{y})_i / \sigma_i^2$

Solution:

$$X_c = U S V^t \Rightarrow V^t X_c^t \mathbf{y} = V^t V S^t U^t \mathbf{y} = S^t U^t \mathbf{y}$$

$$\Rightarrow z_i = (U^t \mathbf{y})_i / \sigma_i = (V^t X_c^t \mathbf{y})_i / \sigma_i^2$$

So $\mathbf{z}$ and finally $\hat{\mathbf{w}} = V \mathbf{z}$ can be computed from $X_c^t X_c$ and $X_c^t \mathbf{y}$ alone!

Chapter 2

Least squares problems

Least-squares and dimensionality reduction

Least-squares and dimensionality reduction

Given n data points in d dimensions:

$$X = \begin{bmatrix} - & \mathbf{x}_1^t & - \\ - & \mathbf{x}_2^t & - \\ - & \vdots & - \\ - & \mathbf{x}_n^t & - \end{bmatrix} \in \mathbb{R}^{n \times d}$$

Want to reduce dimensionality from d to k . Choose k directions $\mathbf{w}_1, \dots, \mathbf{w}_k$, arrange them as columns in matrix W :

$$W = \begin{bmatrix} | & | & & | \\ \mathbf{w}_1 & \mathbf{w}_2 & \dots & \mathbf{w}_k \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{d \times k}$$

Project $\mathbf{x} \in \mathbb{R}^d$ down to $\mathbf{z} = W^t \mathbf{x} \in \mathbb{R}^k$. How to choose W ?

Encoding–decoding model

The projection matrix W serves two functions:

- **Encode:** $z = W^t x$, $z \in \mathbb{R}^k$, $z_j = w_j^t x$.
 - The vectors w_j form a basis of the projected space.
 - We will require that this basis is orthonormal, i.e. $W^t W = I$.
- **Decode:** $\tilde{x} = W z = \sum_{j=1}^k z_j w_j$, $\tilde{x} \in \mathbb{R}^d$.
 - If $k = d$, the above orthonormality condition implies $W^t = W^{-1}$, and encoding can be undone without loss of information.
 - If $k < d$, the decoded $\tilde{x}$ can only approximate x
 $\rightsquigarrow$ the reconstruction error will be nonzero.
- Note that we did not include an intercept term. Assumption: origin of coordinate system is in the sample mean, i.e. $\sum_i x_i = 0$.

Principal Component Analysis (PCA)

We want the reconstruction error $\|\mathbf{x} - \tilde{\mathbf{x}}\|^2$ to be small.

Objective: minimize $\min_{W \in \mathbb{R}^{d \times k}: W^t W = I} \sum_{i=1}^n \|\mathbf{x}_i - W W^t \mathbf{x}_i\|^2$

Finding the principal components

Projection vectors are orthogonal $\leadsto$ can treat them separately:

$$\begin{aligned} & \min_{\mathbf{w}: \|\mathbf{w}\|=1} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{w}\mathbf{w}^t \mathbf{x}_i\|^2 \\ \sum_i \|\mathbf{x}_i - \mathbf{w}\mathbf{w}^t \mathbf{x}_i\|^2 &= \sum_{i=1}^n [\mathbf{x}_i^t \mathbf{x}_i - 2\mathbf{x}_i^t \mathbf{w} \mathbf{w}^t \mathbf{x}_i + \underbrace{\mathbf{x}_i^t \mathbf{w} \mathbf{w}^t \mathbf{w} \mathbf{w}^t \mathbf{x}_i}_{=1}] \\ &= \sum_i [\mathbf{x}_i^t \mathbf{x}_i - \mathbf{x}_i^t \mathbf{w} \mathbf{w}^t \mathbf{x}_i] \\ &= \sum_i \mathbf{x}_i^t \mathbf{x}_i - \sum_i \mathbf{w}^t \mathbf{x}_i \mathbf{x}_i^t \mathbf{w} \\ &= \sum_i \mathbf{x}_i^t \mathbf{x}_i - \mathbf{w}^t \left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^t \right) \mathbf{w} \\ &= \underbrace{\sum_i \mathbf{x}_i^t \mathbf{x}_i}_{\text{const.}} - \mathbf{w}^t \mathbf{X}^t \mathbf{X} \mathbf{w}. \end{aligned}$$

Finding the principal components

- Want to maximize $\mathbf{w}^t X^t X \mathbf{w}$ under the constraint $\|\mathbf{w}\| = 1$
- Can also maximize the ratio $J(\mathbf{w}) = \frac{\mathbf{w}^t X^t X \mathbf{w}}{\mathbf{w}^t \mathbf{w}}$.
- Optimal projection $\mathbf{w}$ is the eigenvector of $X^t X$ with largest eigenvalue (compare handout on spectral matrix norm).
- We assumed $\sum_i \mathbf{x}_i = \mathbf{0}$, i.e. the columns of X sum to zero.
 - ~> compute SVD of “centered” matrix X_c
 - ~> column vectors in W are eigenvectors of $X_c^t X_c$
 - ~> they are the principal components.

Eigen-faces [Turk and Pentland, 1991]

- d = number of pixels
- Each $\mathbf{x}_i \in \mathbb{R}^d$ is a face image
- x_{ij} = intensity of the j -th pixel in image i

$$\begin{array}{ccc} \mathbf{x}_i & \approx & WW^t \mathbf{x}_i = W \mathbf{z}_i \\ (X^t)_{d \times n} & \approx & W_{d \times k} (Z^t)_{k \times n} \\ \left(\begin{array}{c|c|c} \text{img}_1 & \dots & \text{img}_n \end{array} \right) & \approx & \left(\begin{array}{c|c|c|c|c} \text{eigen}_1 & \text{eigen}_2 & \text{eigen}_3 & \text{eigen}_4 & \text{eigen}_5 \end{array} \right) \begin{bmatrix} | & & | \\ \mathbf{z}_1 & \dots & \mathbf{z}_n \\ | & & | \end{bmatrix} \end{array}$$

Conceptual: We can learn something about the structure of face images.

Computational: Can use $\mathbf{z}_i$ for efficient nearest-neighbor classification:

Much faster when $k \ll d$.

Information retrieval: Latent Semantic Analysis

[Deerwater, 1990]

- d = number of words in the vocabulary, say 10000.
- Each $\mathbf{x}_i \in \mathbb{R}^d$ is a vector of word counts
- x_{ij} = frequency of word j in document i

$$\begin{array}{c}
 (X^t)_{d \times n} \\
 \left[\begin{array}{ccccc} \text{stocks:} & 2 & \dots & 0 \\ \text{chairman:} & 4 & \dots & 1 \\ \text{the:} & 8 & \dots & 7 \\ \dots & \vdots & \dots & \vdots \\ \text{wins:} & 0 & \dots & 2 \\ \text{game:} & 1 & \dots & 3 \end{array} \right]
 \end{array}
 \approx
 \begin{array}{c}
 W_{d \times k} \\
 \left[\begin{array}{ccc} 0.4 & \dots & -0.001 \\ 0.8 & \dots & 0.03 \\ 0.01 & \dots & 0.04 \\ \vdots & \dots & \vdots \\ 0.002 & \dots & 2.3 \\ 0.003 & \dots & 1.9 \end{array} \right]
 \end{array}
 \approx
 \begin{array}{c}
 (Z^t)_{k \times n} \\
 \left[\begin{array}{ccc} | & & | \\ \mathbf{z}_1 & \dots & \mathbf{z}_n \\ | & & | \end{array} \right]
 \end{array}$$

How to measure similarity between two documents? Dot products $\mathbf{x}_i^t \mathbf{x}_j$

In such high-dimensional spaces most pairs of vectors are almost orthogonal $\rightsquigarrow$ scalar products tend to be “noisy”.

If $k \ll d$, $\mathbf{z}_i^t \mathbf{z}_j$ is probably a better similarity measure than $\mathbf{x}_i^t \mathbf{x}_j$.

Appendix Chapters 1/2

The Gershgorin circle theorem

Gershgorin circle theorem

Every eigenvalue of $A_{n \times n}$ is in one or more of n circles in the complex plane. Each circle is centered at a diagonal entry a_{ii} , the radius is $r_i = \sum_{j \neq i} |a_{ij}| \rightsquigarrow$ “Gershgorin disk” $D(a_{ii}, r_i)$.

Proof: $A\mathbf{v} = \lambda\mathbf{v}$, assume i is the index for which $|v_i| \geq |v_j|$, $\forall j \neq i$

$$\begin{aligned}(A\mathbf{v})_i &= \lambda v_i \quad \Leftrightarrow \quad \sum_j a_{ij} v_j = \lambda v_i \\(\lambda - a_{ii})v_i &= \sum_{j \neq i} a_{ij} v_j \\|\lambda - a_{ii}||v_i| &= \left| \sum_{j \neq i} a_{ij} v_j \right|\end{aligned}$$

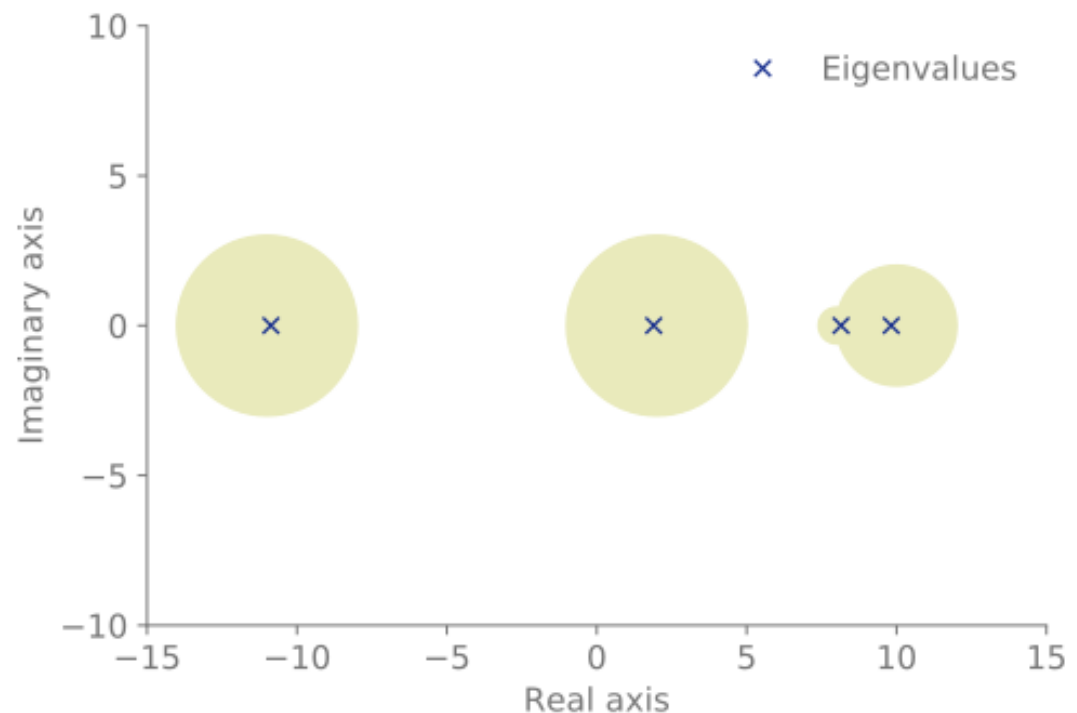
$$\rightsquigarrow \left| \sum_{j \neq i} a_{ij} v_j \right| \leq \sum_{j \neq i} |a_{ij}| |v_j| \leq \sum_{j \neq i} |a_{ij}| |v_i| = r_i |v_i|$$

$$\rightsquigarrow |\lambda - a_{ii}| |v_i| \leq r_i |v_i| \rightsquigarrow |\lambda - a_{ii}| \leq r_i.$$

Applied to A^t : λ_i must also lie within circles corresponding to the columns of A .

Example

$$A = \begin{bmatrix} 10 & -1 & 0 & 1 \\ 0.2 & 8 & 0.2 & 0.2 \\ 1 & 1 & 2 & 1 \\ -1 & -1 & -1 & -11 \end{bmatrix}$$



By Nicoguardo - Own work, CC BY 4.0, <https://commons.wikimedia.org/w/index.php?curid=76601319>

For every row, a_{ii} is the center for the disc with radius $\sum_{j \neq i} |a_{ij}| = r_i$.

Discs: $D(10, 2)$, $D(8, 0.6)$, $D(2, 3)$, $D(-11, 3)$.

Can improve the accuracy of last two discs by applying the formula to the columns: $D(2, 1.2)$ and $D(-11, 2.2)$. True eigenvalues are 9.8218, 8.1478, 1.8995, -10.86 .

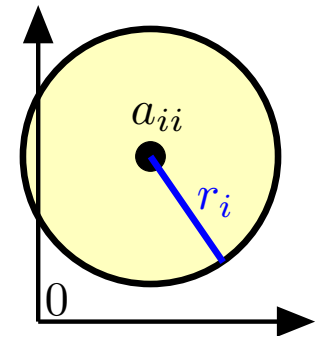
Note that A^t is diagonal dominant: $|a_{ii}| > \sum_{j \neq i} |a_{ji}| \rightsquigarrow$ most of the matrix is in the diagonal $\rightsquigarrow$ explains why the eigenvalues are so close to the centers.

Gershgorin circle theorem and diagonal dominance

A diagonal dominant matrix (i.e. $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$) is **non-singular**.

$\lambda \in \mathbb{C}$ is in at least one of the Gershgorin discs $D(a_{ii}, r_i)$ in the complex plane, but none of these discs contains 0:

$|a_{ii}| - r_i = |a_{ii}| - \sum_{j \neq i} |a_{ij}| > 0$, so each disc center a_{ii} is further away from 0 than the disc radius, and the point $\lambda = 0$ can't belong to any circle.



A **symmetric** diagonal dominant matrix that has **positive** values on its diagonal (i.e. $a_{ii} > \sum_{j \neq i} |a_{ij}|$) is **positive definite**.

Eigenvalues of symmetric matrices are real.

$\lambda \in \mathbb{R}$ is in at least one of the intervals $[a_{ii} - r_i, a_{ii} + r_i]$, but all intervals contain only positive numbers: $a_{ii} - r_i = a_{ii} - \sum_{j \neq i} |a_{ij}| > 0$.

Consequences: Jacobi iterations

- Assume that all diagonal entries of A are nonzero.
- Write $A = D + L + U$

where $D = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix}$ and $L+U = \begin{bmatrix} 0 & a_{12} & \cdots & a_{1n} \\ a_{21} & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & 0 \end{bmatrix}$

- So $Ax = b \rightsquigarrow (L + D + U)x = b$.
- Define $J = D^{-1}(L + U)$ as the **iteration matrix**.
- The solution is then obtained iteratively via

$$x_{(i+1)} = -Jx_{(i)} + D^{-1}b.$$

- Error $\epsilon_{(i+1)} = -J\epsilon_{(i)} = \cdots = (-1)^{i+1}J^{i+1}\epsilon_{(0)}$.
- Arrange eigenvalues of J in diagonal matrix Λ .

Consequences: Jacobi iterations

If all the eigenvalues of J have magnitude < 1 ,
then $\Lambda^n \rightarrow 0$ and consequently $J^n \rightarrow 0 \rightsquigarrow$ convergence.

A diagonally dominant $\rightsquigarrow$ Jacobi method converges.

Assume rows of A are rescaled such that diagonal entries are all 1.

If $A = L + I + U$ is diagonal dominant, i.e. $1 \geq$ row sums of $\text{abs}(L + U)$,
then $L \pm \lambda I + U$ is also diagonally dominant if $|\lambda| \geq 1$,
because $|\lambda| \geq 1 \geq$ row sums of $\text{abs}(L + U)$.

Let λ be an eigenvalue of J .

$$\Rightarrow \det(J - \lambda I) = \det(L + U - \lambda I) = 0.$$

But if $|\lambda| \geq 1$, then $L + U - \lambda I$ is diagonal dominant as well, so it is non-singular and $\det = 0$ is not possible. So $|\lambda| < 1$.